



Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Nikola Dmitrović, Marin Kišić, Dorijan Lendvaj, Martina Licul, Bartol Markovinović, Lovro Nidogon, Patrik Pavić, and Petar Sruk.

Implementation examples are given in attached source code files.

Task Desni klik

Prepared by: Lovro Nidogon

Necessary skills: loops

We need to look at each matrix independently. For each, we have to determine the minimum order number of a row with the character #, and the maximum order number of such row. We can do that with nested loops and helper variables with the currently minimum and maximum order number. We try to update the helper variables every time we are on a field with character #. The needed value for each of the matrices we will get as the difference of the maximum and minimum order number.

Task Čokolade

Prepared by: Vito Anić

Necessary skills: binary search, prefix sums

First, let us observe that the order of chocolates, does not matter, so we can sort them. Equivalent to the value $l - f$ is to sum the value $l - f$ of every chocolate separately. If we need one chocolate, it would be optimal to take the most expensive or the cheapest chocolate. The reason why that is the case is described in the condition of the task:

- If the chocolate is cheaper than k kunas, Lana will pay full price and therefore she would like to take the chocolate with the smallest price possible.
- If the chocolate is more expensive than k kunas, the value $l - f$ for that chocolate will be equal to $k - (c - k) = 2k - c$. So, the more expensive chocolate is, the better.

With this, we can solve the first partial. If we split the chocolates into two categories ($c \leq k$ and $c > k$), we can iterate over the number of cheaper chocolates. If Lana takes x chocolates cheaper than k (first x chocolates in the sorted sequence), she will take $m - x$ expensive (last $m - x$ chocolates). Time complexity is $O(nq)$.

Let us look at this example: Consider a query with $k = 7$ and $m = 4$.

1	2	5	6	8	11	15	20
---	---	---	---	---	----	----	----

Four numbers are larger than 7, and four are smaller. Let us assume that we want to take all four chocolates smaller than k . Can we make the required value smaller? The answer is yes. If we would instead of the chocolate that costs 6 kunas (value 6) take the chocolate that costs 20 kunas (value -6), the total value would be smaller. If we compare the next two possible chocolates from the ends (5 and 15), we can see that 5 has a value of 5, and 15 has a value of -1 , so it is even better to take two of the chocolates from the right end! With that step, we have reached the best solution (1, 2, 15, 20) (because the values of the next two chocolates are 2 and 4, so the chocolate with the price 2 has a smaller value than the chocolate with the price 11). It is enough to binary search the number of chocolates we are taking with the smaller



than k for each query with the same check we were making in the paragraph above. For every query that will be log operations. Total time complexity is $O(q \log(n))$.

Task Berilij

Prepared by: Petar Sruck

Necessary skills: ad-hoc, graph theory, ternary search

Let's define graph G with vertices representing spaceships and edges representing the condition of two spaceships touching externally. Additionally, let the weight of edges be the distance between the circles it connects. Now the task is equivalent with finding non-negative values for the vertices such that for each edge the sum of values of both vertices it connects is equal to the weight of that edge, for which the sum of squares of values of vertices is the least possible. If the vertices (i, j) are connected with the edge of weight $w_{i,j}$ then for the values of vertices the conditions $v_i \geq 0$, $v_j \geq 0$ i $v_i + v_j = w_{i,j}$ hold.

In the first subtask graph G is an odd cycle. Let's assume that, for now, the value of first vertex is 0. Since we can calculate the values $w_{i,j}$ for each edge, we can uniquely determine the value of the second vertex in a cycle. Now that we know the value of the second vertex we can uniquely determine the value of the third, and so on untill we circle back to the first vertex with the condition that it's value should be a . Let's now attempt to increase the value of the first vertex by x . To keep the conditions satisfied we now need to decrease the value of the second vertex, and after that increase the value of the third vertex, and so on untill we circle back to first vertex with the new condition that it's value must be $a - x$. Since $x = a - x$ we can determine x uniquely as $x = \frac{a}{2}$. Now we only need to check if substituting $\frac{a}{2}$ as the value of the first vertex yields non-negative values for all other vertices.

In the third subtask the graph is a forest, but it is enough to solve it for each tree separately. Let's pick any vertex as a root and denote it's value as x . To satisfy the conditions of the task we can now uniquely determine the value of each vertex i as a linear polynomial $\pm x + c_i$, where c_i is a constant with value equal to alternating sum of weights of edges from vertex i to the root. Since every value must be non-negative, vertices with values of $x + c_i$ set lower bound for x , while vertices with values of $-x + c_i$ set upper bound for x . If upper bound is lesser than lower bound there are no solutions. To determine the x for which the sum of squares of values of vertices is the least possible let's sum the squares of linear polynomials for each vertex. The result is a quadratic polynomial $ax^2 + bx + c$. Notice that a is equal to the size of the tree so the quadratic polynomial has a minimum at $x = -\frac{b}{2a}$. Since c is not used in this expression and b is equal to the sum of $-2s_i c_i$ for each vertex where s_i is the sign in front of x in polynomial $\pm x + c_i$, we can calculate $-\frac{b}{2a}$ without squaring any numbers. if $x = -\frac{b}{2a}$ is between lower and upper bound, x is our solution, else for x we take the value of lower or upper bound whichever is closer to $-\frac{b}{2a}$.

For full solution let's solve the task for each component on any spanning tree as in solution for subtask three. Now we only need to analyse cycles. Let's note that in the solution of third subtask the polynomial for each vertex on even depth from the root is $x + c_i$, while for every vertex with odd depth it's $-x + c_i$. Since even cycles connect vertices of different depth parity, they only add conditions of form $(+x + c_i) + (-x + c_j) = w_{i,j}$, in other words $c_i + c_j = w_{i,j}$. c_i and c_j are constants so it is easy to check if this condition is met. Odd cycles connect vertices of same depth parity and add conditions of forms $(\pm x + c_i) + (\pm x + c_j) = w_{i,j}$, in other words $\pm x = \frac{1}{2}(w_{i,j} - c_i - c_j)$ which gives us only one solution for x as in subtask one.

The time complexity of described algorithm is $O(n+m)$. Alternatively, the task could be solved with better implementations of ternary search with complexity $O((n+m) \log(C\epsilon^{-1}))$, where C is maximal absolute value of coordinates $C = 10^4$ and ϵ is the required precision.

Task Iksevi

Prepared by: Dorijan Lendvaj

Necessary skills: number theory, sieve of eratosthenes

We can ignore the left and bottom wall, which allows us to consider points with negative coordinates



without changing the answer. That way, every point which is on a tile's corner is some tile's top corner, some tile's left corner, some tile's bottom corner and some tile's right corner. Without loss of generality, let's consider the points which are some tile's top corner. Let's divide the tiles into even and odd rows. If the length of the diagonal in millimeters is $2k$ where k is a natural number, the center of the tiles in even rows is in points $(2ak, 2bk)$ for all integers a and b , so their top corner is in points $(2ak, (2b+1)k)$ for all integers a and b . The center of the tiles in odd rows is then in points $((2a+1)k, (2b+1)k)$ for all integers a and b , so their top corner is in points $((2a+1)k, (2b+2)k)$ for all integers a and b .

For the first subtask, it's enough to precompute all possible tiles' top corner by considering all possibilities of a , b and k , and for each query we can easily answer how many times that point was a top corner.

For the second subtask, notice that the conditions written in the first paragraph require either x to be equal to k multiplied by an odd number and y to be equal to k multiplied by an even number, or for x to be equal to k multiplied by an even number and y to be equal to k multiplied by an odd number. From there we can easily see that x and y must be divisible by k , so we can solve the second subtask by checking the condition for all k which are divisors of both x and y , i.e. all k which are divisors of $\gcd(x, y)$.

For the third subtask, call the GCD of x and y g . A necessary condition for a fixed k can be further simplified to k being a divisor of g and $\frac{x}{k}$ and $\frac{y}{k}$ having a different parity. Let $\frac{g}{k} = h$, $\frac{x}{g} = x'$ and $\frac{y}{g} = y'$. Then h is a divisor of g , $\frac{x}{k} = hx'$ and $\frac{y}{k} = hy'$. If x' and y' have the same parity, multiplying them with the same number isn't going to change that, so the answer is 0. If they don't, then multiplying them with the same even number will change that (both will become even), and multiplying them with the same odd number won't (both of their parities will remain unchanged). Therefore, we want to find the number of natural numbers h which are divisors of g and which are odd, which is the number of odd divisors of g . Before answering the queries, we can use Eratosthenes's sieve to precompute the number of odd divisors of all number between 1 and 10^7 to easily answer all the queries.

Task Neboderi

Prepared by: Patrick Pavić

Necessary skills: Euclid's algorithm, prefix sums

The first subtask is solvable with brute force, by trying out each possible interval. There are $O(n^2)$ such intervals, and by naively calculating the sum in $O(n)$ and the greatest common divisor in $O(n \ln MAX)$, where MAX is the largest height of a skyscraper, it is possible to achieve a complexity of $O(n^3 \ln MAX)$ which is enough to solve this subtask.

Let the function $S(l, r) := h_l + h_{l+1} + \dots + h_r$ be the sum of heights of all skyscrapers inside an interval, and let $\gcd(l, r) := G(h_l, h_{l+1}, \dots, h_r)$ be their greatest common divisor. Notice that the beauty of an interval $[l, r]$ is exactly $S(l, r) \cdot G(l, r)$. For the solution of the second subtask, notice the following relations: $S(l, r) = S(l, r-1) + h_r$ and $G(l, r) = \gcd(G(l, r-1), h_r)$. By that relation it is possible to calculate all values of the functions S and G in complexities $O(n^2)$ and $O(n^2 \ln MAX)$ respectively. That is enough to solve the second subtask.

For the full solution, you should notice the following. For the function S , the following also holds for $l > 1$, $S(l, r) = S(r, 1) - S(l-1, 1)$. It is enough to calculate all values of $S(1, r)$ in $O(n)$ and that is enough to be able to calculate the value of all $S(l, r)$ in $O(1)$. Let us now fix the right border r of the interval we will look at. Notice the following holds for each $l < r$: $G(l, r) = G(l+1, r)$, or $G(l, r) \mid G(l+1, r)$ which implies $2G(l, r) \leq G(l+1, r)$. Now notice that the function $G(l, r)$ can achieve at most $\log MAX$ different values for a fixed r .

Now, notice that for a fixed l , $G(l, r) = G(l+1, r)$ implies $G(l, r+1) = G(l+1, r+1)$. It follows that it is enough to maintain the leftmost l which achieves each possible value of G for the current border r . Because there are at most $O(\log MAX)$ of them, it is possible to iterate through all of them and calculate their beauties. When switching the right border from r to $r+1$, it is enough to try and add the border $r+1$ for the interval $[r+1, r+1]$, and check for each of the current left borders whether the inequality



$G(l, r) > G(l - 1, r)$ still holds for $G(l, r + 1) > G(l - 1, r + 1)$. This can be done using Euclid's algorithm. This gives us an algorithm of the complexity $O(n \log MAX \ln MAX)$ which is enough for full score. Notice that there exist similar algorithms with the same complexity which have a far worse constant. Such implementation were supposed to get points on the third and fourth subtask.